

Evolving Controllers for Mario AI Using Grammar-based Genetic Programming

João Marcos de Freitas, Felipe Rafael de Souza and Heder S. Bernardino

Universidade Federal de Juiz de Fora, Juiz de Fora, MG, Brazil

joao@ice.ufjf.br, feliperafael@ice.ufjf.br, heder@ice.ufjf.br

Abstract—Video games mimic real-world situations and they can be used as a benchmark to evaluate computational methods in solving different types of problems. Also, machine learning methods are used nowadays to improve the quality of non-player characters in order (i) to create human like behaviors, and (ii) to increase the hardness of the games. Genetic Programming (GP) has presented good results when evolving programs in general. One of the main advantage of GP is the availability of the source-code of its solutions, helping researchers to understand the decision-making process. Also, a formal grammar can be used in order to facilitate the generation of programs in more complex languages (such as Java, C, and Python). Here, we propose the use of Grammar-based Genetic Programming (GGP) to evolve controllers for Mario AI, a popular platform to test video game controllers which simulates the Nintendo's Super Mario Bros. Also, as GP provides the source-code of the solutions, we present and analyze the best program obtained. Finally, GGP is compared to other techniques from the literature and the results show that GGP find good controllers, specially with respect to the scores obtained on higher difficulty levels.

Index Terms—game controller, mario ai, grammar-based genetic programming

I. INTRODUCTION

Video games became widely popular recently and computational intelligence methods can be used to improve them. It is not hard to find papers in the literature where good solutions are provided when one desires to obtain a controller that (i) mimics a human player [1] and (ii) defeats its opponents [2]. In addition, a video game is an interesting environment to evaluate computational intelligence techniques as they are able to mimic real-world situations.

A common way to control non-playable characters is using Finite State Machines. However, the number of states (and state-transitions) increases with the complexity of the game. Behavior Tree is an option to solve this issue [3], as it can define the basic behavior of the game and, when the complexity of the games increases, linking trees are used. An approach for finding these trees is presented in [3], where genetic programming is used to evolve controllers that can perform as an adversary or a substitute for a player.

Another approach suitable for designing video game controllers are Artificial Neural Networks (ANNs) [4]. Given a state of the game, ANNs can determine good actions to be executed [5].

Also, ANNs can be used to mimic human players in video games. For instance, in [1] ANN is used to develop a controller

for a fighting game. The results showed that the controller created by ANN is enjoyable to play with.

Players and video game specialists are also interested in understanding the actions performed by these controllers. However, despite the good results achieved by ANNs, it is hard (or even impossible) to understand their decision making process. The same difficult is observed with respect to other machine learning techniques. Thus, the study of methods with the capability of providing good and understandable solutions is important.

In this way, Genetic Programming (GP) emerges as a branch of computer science to automatically generate programs. Inspired by the theory of evolution by natural selection, GP has presented good results in the literature [6]–[9]. GP is mainly used for knowledge discovery, given its capacity of providing the source-code of its solutions. Thus, it is possible to read and, potentially, to understand how the generated programs work.

HyperGP [6] is an adaptation of HyperNEAT [10], which uses Hypercube-based NeuroEvolution of Augmented Topologies using NEAT to evolve large scale ANNs. In [6], a standard GP evolves ANNs to be used as controllers for autonomous mobile agents on a simulated environment. HyperGP found better solutions than those obtained by HyperNEAT when solving that problem.

Mario AI Championship has four tracks to evaluate artificial intelligence (AI) methods [11]: gameplay, learning track, level generation and Turing Test for evaluating human-like behavior. Thus, the capacity of techniques in generating video game controllers can be evaluated by means of Mario AI.

The use of a HyperGP approach is proposed in [12] to evolve Simple Recurrent Networks (SRNs) for a modified version of Markus Perssons Infinite Mario called Mario AI, a simulator of the popular Nintendo's Super Mario Bros. The results obtained by HyperGP were compared to those found by a baseline SRN and a Multi-Layer Perceptron (MLP). HyperGP achieved the best scores in most of the tested cases, specially in the hardest levels considered in that work.

Grammar-based Genetic Programming (GGP) is a GP variant in which a formal grammar is used to constraint the generation of new solutions. As a result, no syntactically invalid solution is created in GGP, making it easy to work with complex languages. Also, domain knowledge can be included to guide the search [13]. It is not hard to find in the literature the successful application of GGP to solve problems from

different areas of knowledge [8], [14], [15].

Grammatical Evolution (GE) is a GGP method, and the application of GE to evolve behavior trees for the Mario AI Championship is proposed in [16]. That method uses blocks of instructions, and the candidate solutions are controllers that select which block must be executed according to the game state. That work showed that GP like techniques are capable of generating competitive controllers and that the use of a grammar has a positive impact on the evolutionary process.

The application of GGP to generate a controller in symbolic form for Mario AI is the main contribution of this work. In order to use GGP, a formal grammar is also proposed. The solution obtained here is written in the simulator's language and, differently from [16], the definition of blocks of instructions is not required. It is important to highlight that the controller found by the proposed approach reach the highest score in the hardest level tested here when compared to other techniques from the literature. Also, the best program found by the proposed GGP is presented, becoming possible to understand the decision-making process and to learn playing Super Mario Bros.

The remaining of the paper is as follows. The Mario AI simulator is presented in Section II. Section III presents GGP while Section IV describes the proposed GGP approach. Computational experiments can be shown in Section V and Section VI concludes the paper and presents some future works.

II. MARIO AI

The Mario AI Championship occurred during some conferences related to computational intelligence and games. This competition encourages people to study computational techniques with the objective of creating controllers for the Infinite Mario simulator, that implement a version of the well known Nintendo's Super Mario Bros.

In the Infinite Mario simulator¹ the player controls Mario through a random generated two dimensional landscape (Super Mario Bros is classified as a platform game) that contains elevations, gaps and blocks of different types. Mario interacts with enemies and the difficulty of the game presents different levels in the simulator. Mario has three states: small, normal and Fire Mode; in this last case, the character is able to throw fireballs. From a position, the character can move right, left, down (dock), and up. Also, Mario can speed up, jump and throw a fireball, and the actions it is able to perform depend on its state. The goal of the game is to survive and go as far as possible. Other elements are present in the game, such as: coins, a mushroom (it changes Mario from small to normal), and a Fire Flower (small and normal Mario are modified to Fire Mode).

The score of the original Super Mario Bros is computed considering the interactions between Mario and other game elements. On the other hand, in [12] the score in Mario AI is defined by how far Mario is able to go in the game.

¹<http://julian.togelius.com/mariocompetition2009>

Considering a given difficulty level, this score computation represents the capacity of a given controller of reaching the game objective: to conclude all the game's levels. Here, we adopt the same strategy from [12].

The simulator provides to the controller two data matrices which may be used in the decision making process: one matrix contains information regarding the map, while the enemies are represented in the other one. In both matrices, Mario is placed on the center of a 22×22 grid.

Normally, the computational intelligence techniques use sub-matrices of these matrices. For instance, 3×3 , 5×5 and 7×7 sub-matrices, centered in Mario character, are employed by HyperGP, SRN and MLP in [12] for conduct their controllers. These sub-matrices are also used here.

III. GRAMMAR-BASED GENETIC PROGRAMMING

Koza [17] is a pioneer in the study of Genetic Programming (GP), a bio-inspired meta-heuristic designed for generating programs. GP is an specialization of Genetic Algorithm (GA); in GP, the candidate solutions represent computer programs. The evolutionary process is inspired by the Darwinian principle of natural selection, where the fittest individuals are most likely to produce offspring.

One of the main advantages of GP is the interpretability of its models. Several machine learning techniques do not present this feature. In addition, we intend in this work to create not only a good controller for Mario AI, but we also desire to understand the decision making process. Therefore, GP techniques in general, and GGP in particular, is a suitable method for this problem.

A program can be represented in GP using many data structures, such as string, graph, and tree [18]. Trees are commonly adopted in GP for this purpose and here, as a grammar-base variant of GP is employed, the candidate programs are encoded as derivation trees. In the Grammar-based Genetic Programming (GGP) techniques, the target language is specified by a formal grammar and all the created candidate programs are syntactically correct. Thus, initialization, crossover and mutation of GGP are designed in order to always generate valid programs.

GGP starts generating the initial population at random. The population is evaluated, for example, by measuring the accuracy of the individuals with respect to the expected values (when solving supervised learning problems) or by calculating a score (when the problem involves video games). Then individuals are selected to be recombined and mutated; a tournament is used here as the selection procedure. The new individuals (generated by crossover and mutation) are evaluated and the current population is replaced by the new one; due to elitism, the best individuals are directly copied from the current population to the next generation. This process is repeated until a given stop criteria is reached, such as an acceptable fitness value or a maximum computational budget. This process is shown in the Algorithm 1.

A standard GP may generate unfeasible solutions (uncompilable source-code, for example) when the problem involves

Algorithm 1: Pseudo-code for Genetic Programming.

```
1 Create a random population  $P$ ;  
2 Evaluate  $P$ ;  
3 while Stop criterion not reached do  
4   while  $|P_{tmp}| < |P|$  do  
5     Select  $p_1 \in p_2$  by tournament;  
6     Copy  $p_1$  to  $p'_1$  and  $p_2$  to  $p'_2$ ;  
7     if  $random \leq p_{crossover}$  then  
8       Crossover( $p'_1, p'_2$ );  
9     if  $random \leq p_{mutation}$  then  
10      Mutate( $p'_1$ );  
11      Mutate( $p'_2$ );  
12      Evaluate  $p'_1$  and  $p'_2$ ;  
13      Insert  $p'_1$  and  $p'_2$  in  $P_{tmp}$ ;  
14    $P \leftarrow$  elite from  $P \cup$  elite from  $P_{tmp}$ ;  
15   discard  $P_{tmp}$ ;  
16 return best individual;
```

a more complex language. GGP uses a formal grammar to constraint the language of the programs generated during the search. Thus, only syntactically valid solutions are produced in GGP.

A. Context-free Grammar

A formal grammar can be defined by the quadruple $G = (N, \Sigma, S, P)$ [19], where:

- N is the set of non-terminals, which are auxiliary elements of the grammar;
- Σ is the set terminals, which are the elements present in the words of the language;
- S is the start symbol ($S \in N$); and
- P is the set of production rules.

In the context-free grammars like that used here, the productions in P are in the form

$$\langle NT \rangle ::= prod_1 \mid \dots \mid prod_{np}$$

where $prod_i$, $i = 1, \dots, np$, is a production allowed for the non-terminal $\langle NT \rangle \in N$, and $prod_i$ is formed by a combination of terminals and non-terminals (including the empty set).

An example of P designed to generate arithmetic expressions using one- and two-argument functions is

$$\begin{aligned} \langle expr \rangle &::= \langle expr \rangle \langle expr \rangle \langle op \rangle \\ &\quad \mid \langle expr \rangle \langle uop \rangle \\ &\quad \mid \langle var \rangle \mid \langle const \rangle \\ \langle op \rangle &::= + \mid - \mid \times \mid \div \mid pow \\ \langle uop \rangle &::= log \mid exp \mid \sqrt{} \mid sin \mid cos \mid tg \\ \langle var \rangle &::= x \mid y \mid z \\ \langle const \rangle &::= 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \mid \pi \end{aligned}$$

In this example, the one-argument function ($\langle uop \rangle$) are exponential (exp), logarithm (log), square root ($\sqrt{}$), sine (sin), cosine (cos) and tangent (tg), while the two-argument functions ($\langle op \rangle$) are $+$, $-$, $\div$, $\times$, and pow. One can notice that this illustrative grammar uses a post-fix notation to preserve the precedence of operations and the recognized expressions can be evaluated by a stack. Thus, this grammar generates “ $x \ 3 \ +$ ” instead of “ $x + 3$ ”, its infix representation.

When using a formal grammar in a tree-encoded GP, the candidate programs are represented using derivation trees. In a derivation tree, the start symbol S is the root node, the interior nodes are non-terminals from N , and the leaf nodes are terminals from Σ . The production rules of the grammar (P) define the sequence of the elements in the derivation tree.

B. Genetic Operators

The crossover and mutation operators of GGP perform similarly to those from the standard tree-based GP. However, in GGP the genetic operations use a formal grammar to ensure that only syntactically correct models are generated.

The crossover procedure generate two new programs from two selected parent individuals. Sub-trees are swapped between the parents to generate the offspring. In GGP, the sub-tree's root of the second parental must be equals to the sub-tree's root of the first one. Thus, the syntax of the new programs (offspring) is correct. Figure 1a illustrates the application of crossover to individuals A and B . A sub-tree from A is randomly selected and, after that, a sub-tree with the same root is randomly selected in B . These randomly chosen sub-trees are swapped and two new individuals (A' and B') are created.

The mutation operator uses the formal grammar to modify the individuals generated by crossover. In mutation, a sub-tree is randomly selected and it is replaced by another one randomly generated. This new sub-tree is created using the same root (non-terminal) of the original one. Figure 1b illustrates the mutation of the individual B' .

IV. THE PROPOSED APPROACH

We are interested here on (i) the adaptation of GGP to create a controller for Mario AI, (ii) the analysis of the performance of GGP when solving this video game problem, and (iii) the extraction of useful knowledge from the solution obtained by GGP. In order to develop a controller for Mario AI, we propose a context-free grammar.

The Mario AI simulator uses a class implemented in Java to control Mario. In this work, the goal of GGP is to generate a controller in the form of a computer program in Java. Thus, the candidate program is incorporated to Mario AI, the simulator is executed, and its score can be computed. In the proposed approach, the fitness value is the score returned by the simulator.

As the controller of the player is a Java code, the language defined by the proposed grammar is a sub-language of the Java language. Also, some variables and functions of the simulator are added to the grammar. For instance, the function

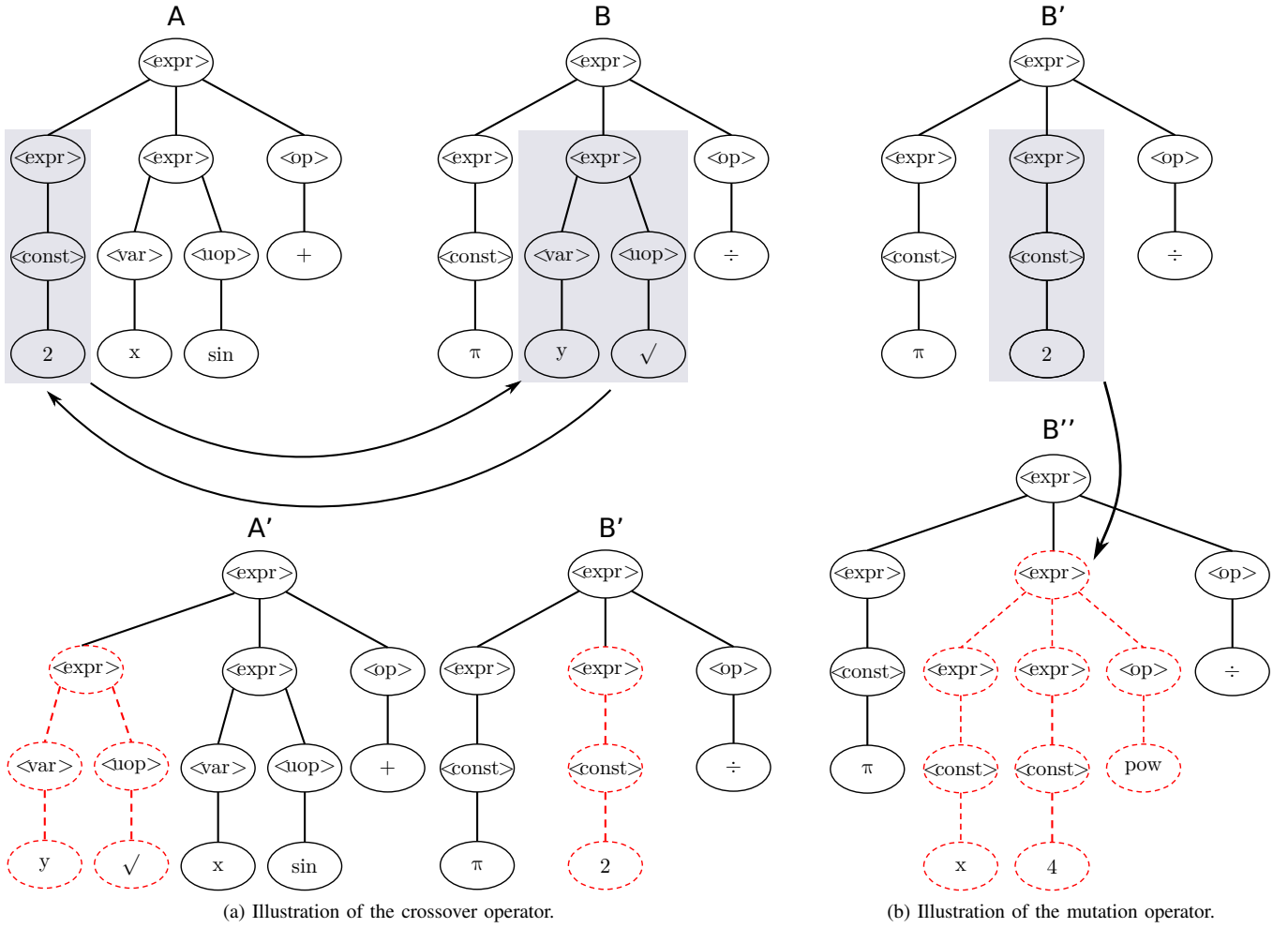


Fig. 1. Examples of the crossover and mutation operators of GGP.

mayMarioJump() returns true if Mario can jump, and false, otherwise. It is important to highlight that Mario AI allows for the player to jump (to the contrary direction) when the character is leaning against a wall. The function *isMarioOnGround()* returns true if Mario is on the floor, and false, otherwise.

The proposed context-free grammar to obtain a controller for Mario AI is presented in Figure 2. As previously presented, some variables and functions of the simulator are in the grammar and, thus, they are valid instructions. The matrix *landscape* contains elements which correspond to game objects. These game objects refer to a map component, such as floor, box, tube, and empty. The matrix *enemies* contains values which represent enemies, such as goomba, bullet, and koopa. These matrices are centered at the position (11,11), which is the cell where Mario is placed.

As in [12], we allow the controller to access only sub-matrices of *landscape* and *enemies*. The three fields of view considered in this work are presented in Figure 3 and are defined as:

- Mario can see only one cell around him. In this case,

the matrices are 3×3 and they are highlighted with blue color in the figure;

- Mario can see two cells around him. In this case, the matrices are 5×5 and they are highlighted with blue and yellow colors in the figure;
- Mario can see three cells around him, the matrices are 7×7 , and they are highlighted with blue, yellow and red colors in the figure;

One can notice that the proposed grammar presented in Figure 2 is used by GGP with 3×3 sub-matrices. Thus, the proposed grammar must be adapted when using 5×5 or 7×7 sub-matrices. In these two other cases, the productions of the non-terminal *<position>* are replaced as:

- 5×5 sub-matrices:
 $\langle position \rangle ::= 9 \mid 10 \mid 11 \mid 12 \mid 13$
- 7×7 sub-matrices:
 $\langle position \rangle ::= 8 \mid 9 \mid 10 \mid 11 \mid 12 \mid 13 \mid 14$

V. COMPUTATIONAL EXPERIMENTS

Computational experiments are performed here in order to evaluate GGP when generating programs to control Mario in

```

< expr > ::= if(< condition >)< expr >
          | if(< condition >)< expr >else< expr >
          | < expr >< expr > | < action >
< condition > ::= enemies[< pos >][< pos >] < comp > < enemies >
               | landscape[< pos >][< pos >] < comp > < landscape >
               | observation.isMarioOnGround()
               | observation.mayMarioJump()
< action > ::= action[< key >] = < bool >;
< compare > ::= == | !=
< key > ::= Mario.KEY_LEFT | Mario.KEY_RIGHT | Mario.KEY_DOWN
          | Mario.KEY_JUMP | Mario.KEY_SPEED
< bool > ::= true | false
< enemies > ::= Sprite.KIND_GOOMBA | Sprite.KIND_GOOMBA_WINGED
              | Sprite.KIND_RED_KOOPA | Sprite.KIND_RED_KOOPA_WINGED
              | Sprite.KIND_GREEN_KOOPA | Sprite.KIND_GREEN_KOOPA_WINGED
              | Sprite.KIND_BULLET_BILL | Sprite.KIND_SPIKY
              | Sprite.KIND_SPIKY_WINGED
< landscape > ::= 0 | 16 | 21 | -10 | -11 | 20
< position > ::= 10 | 11 | 12

```

Fig. 2. Proposed context-free grammar for GGP when applied to the development of a controller for Mario AI.



Fig. 3. Data matrices *landscape* and *enemies*. The sizes considered here for these matrices are 3×3 , 5×5 , and 7×7 .

the Mario AI simulator. The solutions obtained by GGP are compared to those found by HyperGP, SRN, and MLP [12]. Also, the best controller achieved by GGP is analyzed.

The parameter settings used here for GGP are presented in Table I. The number of objective function evaluations allowed is equal to 10,000; this is the same computational budget used in [12]. Additionally, 30 independent runs were performed for

TABLE I
PARAMETERS USED ON GGP

Parameter	value
Population Size	100
Generations	100
Crossover Prob	90%
Mutation Prob	90%
Tree depth	10

each grammar.

The training protocol is the same used in [12]: (i) the evolution starts on the difficulty level 0; (ii) all the individuals are evaluated using the same level to calculate their fitness/score; and (iii) when one of the controllers wins the current difficulty level or reaches a score greater than or equals to 4,000, the difficulty level is increased by one and all the individuals of the population are evaluated with respect to the same (harder) level. So, every time a controller win a level, the evolutionary environment changes to a harder level with new obstacles and enemies.

The best solution obtained in each independent run (training phase) is also evaluated using other maps (testing phase). These best controllers are executed using 4,000 maps of difficulty levels 0, 3, 5 and 10; 1,000 times on each level. Those testing maps are the same for every independent run performed, making it possible to compare the results found.

TABLE II
RESULTS OBTAINED BY GGP – 3×3 SUB-MATRICES.

Level	min	median	average	std	max
maxLevel	0.00	1.00	1.07	1.08	3.00
score					
0	347.29	3920.36	2985.35	1536.30	4277.60
3	269.13	1413.38	1276.39	610.31	2036.34
5	273.69	1221.28	1096.25	498.57	1760.39
10	250.20	955.90	877.18	379.04	1358.89
number of wins					
0	0.00	804.00	586.81	413.01	980.00
3	0.00	20.00	33.26	35.47	101.00
5	0.00	3.00	9.81	12.63	37.00
10	0.00	0.00	1.52	2.22	7.00

TABLE III
RESULTS OBTAINED BY GGP – 5×5 SUB-MATRICES.

Level	min	median	average	std	max
maxLevel	0.00	2.00	1.37	1.02	4.00
score					
0	240.72	3728.96	2563.32	1737.17	4279.07
3	243.60	1247.61	1100.70	656.30	2033.42
5	272.51	1116.98	953.15	528.58	1722.87
10	246.85	936.82	770.94	396.98	1368.20
number of wins					
0	0.00	741.00	498.26	443.62	972.00
3	0.00	10.00	25.26	33.43	98.00
5	0.00	2.00	8.07	12.82	37.00
10	0.00	0.00	0.85	2.12	10.00

Tables II, III and IV show the results found by GGP. The difficulty level at the end of the training phase is represented by “maxLevel”. Scores and number of wins are presented for each level tested here, namely, 0, 3, 5 and 10.

Table V shows the results found by GGP and those presented in [12] when MLP, SRN and HyperGP were used. In this table, the superscripts represent the size of the *landscape* and *enemies* matrices (3×3 , 5×5 or 7×7). The best values are highlighted in boldface.

All experiments were conducted on a Intel Core i7-3770 CPU @ 3.40GHz x 8 with 8GB of RAM and with a Linux operational system. GGP was implemented using C++ programming language and the source-code is available².

²<https://github.com/Joaoufjf/marioai>

TABLE IV
RESULTS OBTAINED BY GGP – 7×7 SUB-MATRICES.

Level	min	median	average	std	max
maxLevel	0.00	1.00	1.07	0.98	3.00
score					
0	345.80	2882.72	2598.68	1665.27	4280.64
3	268.11	1327.34	1115.68	596.70	2015.82
5	276.18	1145.68	971.93	481.84	1720.09
10	245.35	911.22	795.34	365.72	1371.33
number of wins					
0	0.00	434.00	491.30	437.88	972.00
3	0.00	11.00	22.93	31.13	92.00
5	0.00	1.00	6.81	11.59	35.00
10	0.00	0.00	0.96	1.86	7.00

TABLE V
AVERAGE OF THE HIGHEST DIFFICULT LEVEL AND AVERAGE OF THE SCORES OBTAINED IN THE LEVELS 0, 3, 5 AND 10 WHEN USING MLP, SRN, HYPERGP, AND THE PROPOSED GGP. THE SUPERSCRIPTS ARE THE SIZE OF THE *landscape* AND *enemies* MATRICES.

Controller	Level	0	3	5	10
MLP ^{3×3}	3.00	1784.00	719.00	606.00	531.00
MLP ^{5×5}	0.83	864.00	456.00	410.00	377.00
MLP ^{7×7}	0.50	559.00	347.00	345.00	300.00
SRN ^{3×3}	2.83	3050.00	995.00	834.00	690.00
SRN ^{5×5}	1.83	1625.00	670.00	576.00	512.00
SRN ^{7×7}	0.25	757.00	440.00	408.00	373.00
HyperGP ^{3×3}	1.87	2365.00	780.00	695.00	585.00
HyperGP ^{5×5}	2.13	2352.00	786.00	679.00	545.00
HyperGP ^{7×7}	1.25	2314.00	633.00	588.00	534.00
GGP ^{3×3}	1.07	2985.35	1276.39	1096.24	877.17
GGP ^{5×5}	1.37	2563.32	1100.70	953.15	770.94
GGP ^{7×7}	1.07	2598.68	1115.68	971.92	795.34

A. Analysis of the Results

One can see in Tables II, III and IV that the average score values are better when the smallest sub-matrices (3×3) are used. This result reflects on the number of wins, where GGP with 3×3 sub-matrices reached more wins, on the average, than the other variants in all difficulty levels. On the other hand, GGP with 5×5 sub-matrices won 10 times the difficulty level 10 while the other variants reached only 7 wins. In general, the three approaches performed similarly.

Table V shows that MLP^{3×3} reached the highest difficulty level on average. When only the GP techniques are considered (GGP and HyperGP) the highest level in average is obtained by HyperGP^{5×5}.

GGP^{3×3} achieved the highest scores with respect to the average for the levels 3, 5, and 10. When the difficulty level is 0, the technique with the highest average score is SRN^{3×3}. Thus, one can see that the best results on average are found when using 3×3 matrices. Among the GP techniques, GGP^{3×3} outperforms the other methods with respect to the average score values.

B. Complementary Analysis

As highlighted before, one of the main advantage of GGP is the symbolic form of the programs in arbitrary language. This feature helps the user to understand the decision making process of the obtained program. Also, with the source-code of the solution, one can create even better controllers.

Figure 4 presents the source-code of the best controller with respect to the score obtained in the difficulty level 10. The score of this model is equal to 1371.33. One can notice that despite the use of 7×7 sub-matrices in the formal grammar, it uses only the indexes 10, 11 and 12.

According to the source-code of the best controller found by GGP, the initial state in the vector “action” is false for every button. In lines 15-34 the controller looks for enemies, going forward or avoiding docking. The initial lines prevent Mario to do consecutive low jumps, always running before the jumps.

```

1 void getAction() {
2     byte[][] enemies = observation.getEnemiesObservation();
3     byte[][] landscape = observation.getLevelSceneObservation();
4     if (landscape[12][12] != -11) {
5         if (observation.mayMarioJump()) {
6             action[Mario.KEY_JUMP] = true;
7         } else {
8             if (observation.isMarioOnGround()) {
9                 action[Mario.KEY_JUMP] = false;
10            }
11        }
12    } else {
13        action[Mario.KEY_SPEED] = true;
14    }
15    if (landscape[10][10] != 16) {
16        action[Mario.KEY_SPEED] = true;
17    }
18    if (landscape[10][12] != 21) {
19        if (enemies[11][12] != Sprite.KIND_SPIKY) {
20            if (observation.isMarioOnGround()) {
21                if (enemies[10][11] != Sprite.KIND_GOOMBA) {
22                    if (enemies[11][10] != Sprite.KIND_SPIKY_WINGED) {
23                        action[Mario.KEY_RIGHT] = true;
24                    }
25                }
26            } else {
27                if (enemies[12][10] != Sprite.KIND_BULLET_BILL) {
28                    if (observation.mayMarioJump()) {
29                        action[Mario.KEY_DOWN] = false;
30                    }
31                } else {
32                    if (observation.mayMarioJump()) {
33                        action[Mario.KEY_SPEED] = false;
34                    }
35                }
36            }
37        }
38    } else {
39        action[Mario.KEY_DOWN] = true;
40    }
41 }

```

Fig. 4. Controller of the best result found by GGP in the difficult level 10. The method *getAction()* is called by the simulator to perform the action defined by this controller.

Additionally, two independent runs with a budget equals to 100,000 objective functions evaluations were performed. The first run evolved 50 individuals within 2,000 generations, while the second one used 100 individuals and 1,000 generations. Here, 7×7 sub-matrices are used and the parameters are those adopted previously.

As expected, GGP generated better results when more computational budget is allowed. Both the tested cases reached the difficulty level 5. The score of the best solutions at the end of the search are: 2442.71, when evolving 100 individuals, and 1882.50, when 50 individuals are evolved. According to these results, GGP with 100 individuals obtained the best fitness value in the difficulty level 5.

VI. CONCLUDING REMARKS AND FUTURE WORKS

The problem of developing controllers for video games using a Grammar-based Genetic Programming (GGP) is studied here. GGP normally uses a context-free grammar in order to generate the valid programs. With the source-code of the solution, one can understand the decision making process and extract knowledge from it.

We proposed here a context-free grammar for GGP to evolve controllers for the Mario AI simulator. According to the results obtained in the computational experiments, we conclude that (i) GGP has a competitive performance against other techniques from the literature; (ii) GGP found controllers with the highest scores in the most difficulty levels; (iii) GGP with 3×3 sub-matrices of enemies and map elements

outperforms all the GP techniques considered here; and (iv) for all the tested techniques, best results are obtained when using 3×3 sub-matrices. Also, the source-code of the best solution found by GGP is presented and some strategies for playing Super Mario were extracted.

Finally, as previously presented, the difficulty level increases when a controller win the current level and a new map is used to evaluate the candidate solutions of the population. This makes the landscape of the search space dynamic, normally leading to a harder optimization problem. Thus, the evaluation of dynamic optimization strategies when combined with GGP approaches is an interesting research avenue. Other future works are (i) to modify the training protocol; (ii) to explore other objective functions of the game, such as the number of coins collected and the number of enemies defeated.

ACKNOWLEDGES

The authors thank the reviewers for their comments, and the financial support provided by FAPEMIG (grants APQ-03414-15 and PCE-00439-18), CNPq, PPGMC, and PPGCC.

REFERENCES

- [1] M. R. Mendonça, H. S. Bernardino, and R. F. Neto, "Simulating human behavior in fighting games using reinforcement learning and artificial neural networks," in *Proc. of the Brazilian Symposium on Computer Games and Digital Entertainment (SBGames)*, 2015, pp. 26–32.
- [2] —, "Reinforcement learning with optimized reward function for stealth applications," *Entertainment Computing*, vol. 25, pp. 37 – 47, 2018.
- [3] C.-U. Lim, R. Baumgarten, and S. Colton, "Evolving behaviour trees for the commercial game defcon," in *Applications of Evolutionary Computation*, C. Di Chio, S. Cagnoni, C. Cotta, M. Ebner, A. Ekárt, A. I. Esparcia-Alcázar, C.-K. Goh, J. J. Merelo, F. Neri, M. Preuß, J. Togelius, and G. N. Yannakakis, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 100–110.
- [4] N. Rochester, J. Holland, L. Haibt, and W. Duda, "Tests on a cell assembly theory of the action of the brain, using a large digital computer," *IRE Transactions on information Theory*, vol. 2, no. 3, pp. 80–93, 1956.
- [5] K. O. Stanley, B. D. Bryant, and R. Mäkeläinen, "Evolving neural network agents in the nero video game," *Proceedings of the IEEE*, pp. 182–189, 2005.
- [6] Z. Buk, J. Koutník, and M. Šnork, "Neat in hyperneat substituted with genetic programming," in *Adaptive and Natural Computing Algorithms*, M. Kolehmainen, P. Toivanen, and B. Beliczynski, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 243–252.
- [7] H. Bernardino, "Programaço imunológica gramatical para inferência automática de modelos e projeto timo de estruturas," Ph.D. dissertation, Laboratório Nacional de Computação Científica (LNCC/MCTI), 2012.
- [8] J. de Freitas, H. Bernardino, J. Guerreiro, and H. Barbosa, "Aplicação de uma programação genética gramatical multiobjetivo na inferência da máxima deformação longitudinal de dutos com amassamento," in *Proc. of the Iberian Latin American Congress on Computational Methods in Engineering (CILAMCE)*, 2015.
- [9] R. V. Veiga, J. M. de Freitas, H. S. Bernardino, H. J. C. Barbosa, and N. M. Alcântara-Neves, "Using grammar-based genetic programming to determine characteristics of multiple infections and environmental factors in the development of allergies and asthma," in *2015 IEEE Congress on Evolutionary Computation (CEC)*, May 2015, pp. 1604–1611.
- [10] K. O. Stanley, D. B. D'Ambrosio, and J. Gauci, "A hypercube-based encoding for evolving large-scale neural networks," *Artificial Life*, vol. 15, no. 2, pp. 185–212, 2009, pMID: 19199382. [Online]. Available: <https://doi.org/10.1162/artl.2009.15.2.15202>
- [11] J. Togelius, N. Shaker, S. Karakovskiy, and G. N. Yannakakis, "The mario ai championship 2009-2012," *AI Magazine*, vol. 34, no. 3, pp. 89–92, 2013.
- [12] J. Togelius, S. Karakovskiy, J. Koutník, and J. Schmidhuber, "Super mario evolution," in *Computational Intelligence and Games, 2009. CIG 2009. IEEE Symposium on*. IEEE, 2009, pp. 156–161.
- [13] W. Afzal and R. Torkar, "On the application of genetic programming for software engineering predictive modeling: A systematic review," *Expert Systems with Applications*, vol. 38, no. 9, pp. 11 984 – 11 997, 2011.
- [14] P. S. Ngan, M. L. Wong, K. S. Leung, and J. C. Cheng, "Using grammar based genetic programming for data mining of medical knowledge," *Genetic Programming*, pp. 254–259, 1998.
- [15] N. X. Hoai, R. I. McKay, D. Essam, and R. Chau, "Solving the symbolic regression problem with tree-adjunct grammar guided genetic programming: The comparative results," in *Evolutionary Computation, 2002. CEC'02. Proceedings of the 2002 Congress on*, vol. 2. IEEE, 2002, pp. 1326–1331.
- [16] D. Perez, M. Nicolau, M. O'Neill, and A. Brabazon, "Evolving behaviour trees for the mario ai competition using grammatical evolution," in *Applications of Evolutionary Computation*, C. Di Chio, S. Cagnoni, C. Cotta, M. Ebner, A. Ekárt, A. I. Esparcia-Alcázar, J. J. Merelo, F. Neri, M. Preuss, H. Richter, J. Togelius, and G. N. Yannakakis, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 123–132.
- [17] J. R. Koza, *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. The MIT Press, 1992.
- [18] D. A. Augusto, "Programaço genética multi-populacional e co-evolucionária para classificação de dados," Ph.D. dissertation, Universidade Federal do Rio de Janeiro, COPPE, 2009.
- [19] S. Ginsburg, *Algebraic and automata-theoretic properties of formal languages*. Elsevier, 1976.